

Defects & Inconsistencies Inspection

Time spent during task 1: 60 mints

Defects table

Defect #	Location	Defect type	Classification	Description	Status	Date corrected
1	Operating Environment/ Availability (Page 2)	Ambiguity	Major	"The app must be available while at the grocery store" lacks a measurable Uptime SLA/availability percentage or offline policy.		
2	Other Product Requirements /Usability (Page 3)	Unmeasurability	Minor	"The interface must be highly intuitive, requiring no formal training" is subjective and lacks a quantitative usability metric.		
3	Other Product Requirements /Performance (Page 3)	Inadequacy	Major	Requires loading transactional pages in under 2 seconds, but does not specify network bandwidth assumptions (e.g., 3G/4G/5G).		
4	Product Feature/Content Moderation (Page 2)	Omission	Minor	Features Content Moderation Dashboard but omits functional rules or response times for rejecting/approving recipe submissions.		
5	Other Product Requirements /Security (Page 3)	Omission	Major	Data retention, backup policies, and privacy constraints for user-generated meal calendars and recipes are completely missing from the document.		
6	Assumptions and Dependencies (Page 3)	Overspecification	Minor	The document specifies using a "MySQL database" for backend storage, which overspecifies the technical solution at the requirements stage.		
7	Product Feature/Meal Calendar (Page 2)	Poor structuring	Minor	The non-functional performance requirement for "real-time drag-and-drop updates" is embedded within the functional feature list rather than placed in performance requirements.		

8	Product Overview (Page 1)	Forward reference	Minor	Mentions integration with "External Fitness APIs" in later phases but fails to define the scope, data formats, or refer to any external interface document.		
---	---------------------------	-------------------	-------	---	--	--

Inconsistencies table

#	Location	Inconsistency type	Classification	Description	Status	Date corrected
1	Product Feature (Page 2) S1, S2	Structure Clash	Strong	S1 states the system automatically deducts ingredients, while S2 relies on accurate pantry states that might need manual syncing, creating a tracking mismatch.		
2	Assumptions and Dependencies (Page 2) S8 with Other Product Requirements (Page 3) S9	Structure Clash	Strong	S8 requires a persistent backend database connection, which directly conflicts with S9's mobile store access requirement during potential network loss.		
3	Product Overview/ Product Perspective (Page 1) S1	Designation Clash	Weak	The product overview designates pantry contents as "digital pantry", whereas the feature section labels it "Digital Pantry inventory".		
4	Stakeholder Description/ Stakeholder Summary (Page 2) S6	Terminology Clash	Weak	Refers to "Support Specialist" in stakeholders while S6 mentions a "Ticketing Support System" without clear distinction between automated ticketing and staff roles.		
5	Stakeholder Description (Page 2) with Product Feature (Page 2) S4	Designation Clash	Weak	The stakeholder section designates the personnel reviewing recipes as "Content Moderators", while S4 states that "Admins" review, approve, or reject submissions, blurring role boundaries.		

Other comments/recommendations:

No Comments

Task 2: Documenting conflicts

Time spent during task 2: 120 mints

S1: System automatically deducts ingredients from the digital pantry when a meal is cooked.

S2: Users manually sync and enter pantry states to get smart recipe suggestions.

S3: Interactive Meal Calendar allows users to drag-and-drop meal scheduling.

S4: Admins review, approve, or reject recipe submissions.

S5: Admin Health Monitor tracks database load and active user sessions in real-time.

S6: Ticketing Support System provides a portal for students to submit bug reports.

S7: Responsive Web Platform accessible via both mobile and desktop browsers.

S8: System requires a persistent and constant backend database connection.

S9: Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network.

S10: System interface must be highly intuitive, requiring no formal training.

Statement	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	Total
S1	0	1	1000	0	0	0	0	1000	1	1000	3002
S2	1	0	1000	1000	0	0	0	1000	1	1000	4002
S3	1000	1000	0	0	0	0	0	1000	0	1000	4000
S4	0	1000	0	0	0	0	0	1000	0	0	2000
S5	0	0	0	0	0	1000	0	1000	1	0	2001
S6	0	0	0	0	1000	0	0	0	0	1000	2000
S7	0	0	0	0	0	0	0	0	0	1000	1000
S8	1000	1000	1000	1000	1000	0	0	0	1	0	5001
S9	1	1	0	0	1	0	0	1	0	0	4
S10	1000	1000	1000	0	0	1000	1000	0	0	0	5000
Total	3002	4002	4000	2000	2001	2000	1000	5001	4	5000	28010

Task 3: Conflict Resolution

Time spent during task 3: 120 mins

1. Consider the divergence between the statements (S1, S2) 'System automatically deducts ingredients from the digital pantry when a meal is cooked' and 'Users manually sync and enter pantry states to get smart recipe suggestions'.

Statements are conflicting as one relies on automatic deduction while the other depends on manual entry, which can lead to inventory discrepancies.

Operator	Explanation
Avoiding Boundary Condition	The boundary condition for this strong conflict is 'the system and the user overriding the same inventory data simultaneously'. Avoiding this boundary condition might be achieved by adding a validation step where the system auto-calculates the deduction but requires user confirmation before finalizing.
Weaking Conflicting Statements	Weaken conflicting statements by relaxing the strict "automatic deduction" requirement. Instead of a hard automated update, the system can flag items as "likely depleted" based on cooked meals, prompting the user to sync them.
	Resolved Statement: 'System flags potential ingredient deductions when a meal is cooked, prompting users to manually confirm and sync their pantry state for accurate smart recipe suggestions.'

2. Consider the divergence between the statements (S8, S9) 'System requires a persistent and constant backend database connection' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Statements are conflicting because maintaining a persistent backend database connection is nearly impossible to guarantee in grocery store environments with weak mobile signals, inevitably causing page load times to exceed 2 seconds.

Operator	Explanation
Avoiding Boundary Condition	Avoid the boundary condition of 'network connection loss blocking UI rendering' by implementing local data caching (Progressive Web App mode). This allows the application to load transactional pages locally within 2 seconds.
Weaking Conflicting Statements	Weaken conflicting statements by relaxing the "persistent and constant" constraint. Allow the system to gracefully degrade into an offline mode, queuing database updates to be synchronized when the network is restored.
	Resolved Statement: 'System caches necessary data locally to ensure transactional pages load in under 2 seconds in mobile grocery

	environments, and synchronizes asynchronously with the backend database when the connection is restored.'
--	---

3. Consider the divergence between the statements (S1, S9) 'System automatically deducts ingredients from the digital pantry when a meal is cooked' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Statements are conflicting because executing complex automated database deductions upon meal completion can cause rendering latency, violating the strict 2-second performance requirement.

Operator	Explanation
Specializing Conflict	This conflict can be resolved by specializing the conflict source object of "automatic deductions". Offload the automatic deduction processing from the main UI thread to a background service or message queue.
Weaking Conflicting Statements	Weaken conflicting statements by applying the 2-second load time strictly to the UI rendering, while allowing the backend inventory update to process asynchronously over a longer period.
	Resolved Statement: 'System processes automatic ingredient deductions asynchronously in the background to ensure the primary UI transactional pages load in under 2 seconds.'

4. Consider the divergence between the statements (S5, S9) 'Admin Health Monitor tracks database load and active user sessions in real-time' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Statements are conflicting as continuous real-time queries for database health will consume database resources, degrading the performance for end-users trying to load transactional pages.

Operator	Explanation
Specializing Conflict	This conflict can be resolved by Specializing the Conflict Source by separating the databases. Route all Admin Health Monitor queries to an isolated, read-only database replica so it does not affect the main transactional database.
Drop Lower-Priority Statements	Drop lower-priority statements by relaxing "real-time" monitoring to periodic monitoring (e.g., fetching metrics every 5 minutes), prioritizing end-user transactional performance over instantaneous admin metrics.
	Resolved Statement: 'Admin Health Monitor periodically tracks database load using a read-replica database to ensure student transactional pages consistently load in under 2 seconds.'

Task 4: Conflict Evaluation

Time spent during task 4: 120 mints

Using Weighted matrices for evaluating alternative options for the above documented conflicts.

$$\text{totalScore}(\text{opt}) = \sum (\text{Scores}(\text{opt}, \text{crit}) \times \text{Weight}(\text{crit}))$$

1. Consider the divergence between the statements (S1, S2) 'System automatically deducts ingredients from the digital pantry when a meal is cooked' and 'Users manually sync and enter pantry states to get smart recipe suggestions'.

Evaluation Criteria NFR	Significance Weighting	Validation step before deduction	Flag as likely depleted and prompt sync
Data Accuracy	0.5	0.9	0.7
Usability	0.3	0.7	0.8
Minimal inconvenience	0.2	0.6	0.9
Total	1.0	0.78	0.77

The option "Validation step before deduction" seems to be a better option according to above estimates.

2. Consider the divergence between the statements (S8, S9) 'System requires a persistent and constant backend database connection' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Evaluation Criteria NFR	Significance Weighting	Local data caching (PWA mode)	Offline degraded mode (queue updates)
Fast Response	0.5	0.9	0.7
Data Availability	0.3	0.8	0.6
Less Cost	0.2	0.4	0.8
Total	1.0	0.77	0.69

The option "Local data caching (PWA mode)" seems to be a better option according to above estimates.

3. Consider the divergence between the statements (S1, S9) 'System automatically deducts ingredients from the digital pantry when a meal is cooked' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Evaluation Criteria NFR	Significance Weighting	Asynchronous background processing	Apply 2s limit strictly to UI
Fast Response	0.6	0.9	0.7
Reliability	0.2	0.8	0.7
Less Cost	0.2	0.6	0.9
Total	1.0	0.82	0.74

The option "Asynchronous background processing" seems to be a better option according to above estimates.

4. Consider the divergence between the statements (S5, S9) 'Admin Health Monitor tracks database load and active user sessions in real-time' and 'Transactional pages must load in under 2 seconds, even in mobile grocery environments with poor network'.

Evaluation Criteria NFR	Significance Weighting	Route queries to read-replica database	Periodic monitoring (every 5 mins)
Fast Response	0.5	0.9	0.9
Data Freshness	0.3	0.9	0.5
Less Cost	0.2	0.4	0.9
Total	1.0	0.80	0.78

The option "Route queries to read-replica database" seems to be a better option according to above estimates.

Task 5: Risk Management

Time spent during task 5: 180 mints

1. Risk Identification

a) Component Inspection:

Database Server Failure

Currently, the system relies on a persistent and constant backend database connection, acting as a single node and a single point of failure.

b) Risk Checklist

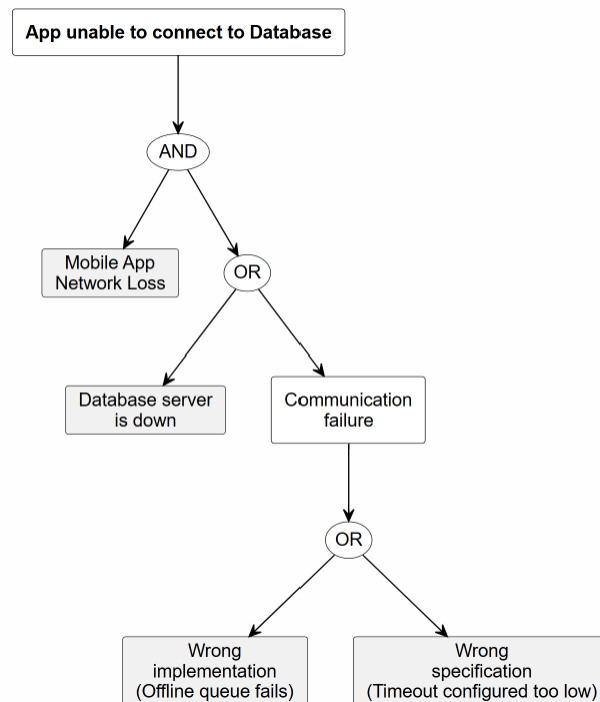
1. **Reliability Risk:** Currently, the system relies on users to accurately and manually log their pantry additions/deletions. If this is forgotten, the smart recipe suggestions become inaccurate and unreliable to trust.

2. **Security Risk:** Currently, the application is not explicitly using any encryption techniques to protect user dietary preferences and meal calendar data. Therefore, it has very high chances of data breaches or hacking.
3. **Performance Risk:** Currently, the system has a risk of low performance when matching extensive pantry inventories against thousands of recipes. This response time needs to be less than 2 seconds, which may be violated under load.

c) Risk Trees

Communication Risk: App unable to connect to database in grocery store environments.

- **[AND]** App unable to connect to Database
 - Mobile App Network Loss
 - **[OR]** Communication failure
 - Database server is down
 - **[OR]** Sync Implementation Failure
 - Wrong implementation (Offline queue fails)
 - Wrong specification (Timeout configured too low)



2. Risk Quantitative Assessment

Risk	Rationale	Likelihood	Severity (1-10)	Risk Exposure (Likelihood*Severity)
Server Failure Risk	Database server is a single point of failure. There are very high chances of server failure in case of heavy user load. (LIKELY)	30%	8	2.4
Reliability Risk	Relying solely on manual user entry for pantry inventory. Possibility of not providing accurate recipe recommendations due to stale data. (POSSIBLE)	20%	6	1.2
Security Risk	There are very high chances of data breaches and hacking if not using any encryption techniques for meal calendars. (LIKELY)	35%	8	2.8
Performance Risk	There is a possibility that generating complex recipes takes longer than the strictly required 2-second threshold. (POSSIBLE)	24%	7	1.68
Communication Risk	There are high chances of communication failure between the mobile App and Database server inside grocery stores. (LIKELY)	40%	8	3.2

3. Risk Control

We will be using Risk Reduction Leverage (RRL) to calculate the better countermeasure for a particular risk.

$$\text{Risk reduction leverage (RRL)} = \frac{\text{RE}_{\text{before}} - \text{RE}_{\text{after}}}{\text{Cost of risk reduction}}$$

$$\text{RE} = \text{risk probability} \times \text{amount at stake}$$

a) Server Failure Risk

Risk Reduction Tactic: Reduce Risk Likelihood by opting below 2 countermeasure alternatives.

Problem Statement: Currently we are using a single database as a single node which is a single point of failure in case of heavy load or due to any other issue. The probability of data loss is 30%. The cost of losing data will be measured in terms of cost of its reproduction which is estimated as 8.

Countermeasure alternative 1 and 2:

1. Add a 3-node database cluster to handle increased traffic and provide replication. So that other server will always be running in case of failure. This option will reduce the risk to 4% and is estimated to cost 3.
2. Monitoring tools could be used to monitor the data and accordingly notification should be sent to the concerned team at the right time (when a particular threshold crossed). This option will reduce the risk to 5% and is estimated to cost 2.

$$RRL_1 = (0.30 \times 8 - 0.04 \times 8)/3 = (2.4 - 0.32)/3 = 0.69$$

$$RRL_2 = (0.30 \times 8 - 0.05 \times 8)/2 = (2.4 - 0.40)/2 = 1.0$$

Since $RRL_2 > RRL_1$ which indicates a cost-effective risk reduction measure. The second alternative is the better option.

b) Reliability Risk

Risk Reduction Tactic: Reduce Consequence Likelihood by opting below 2 countermeasure alternatives.

Problem Statement: Currently we are relying solely on manual user entry for pantry states which is not at all reliable to trust. The probability of not having updated inventory is 20%. The cost of inaccurate data will be measured in terms of recipe generation failure which is estimated as 6.

Countermeasure alternative 1 and 2:

1. Integrate a barcode scanner API to allow quick and accurate scanning of groceries. This option will reduce the risk to 1% and is estimated to cost 1.
2. Prompt users with push notifications daily to review their digital pantry. This option will reduce the risk to 2% and is estimated to cost 2.

$$RRL_1 = (0.20 \times 6 - 0.01 \times 6)/1 = (1.2 - 0.06)/1 = 1.14$$

$$RRL_2 = (0.20 \times 6 - 0.02 \times 6)/2 = (1.2 - 0.12)/2 = 0.54$$

Since $RRL_1 > RRL_2$ which indicates a cost-effective risk reduction measure. The first alternative is the better option.

c) Security Risk

Risk Reduction Tactic: Reduce Risk Likelihood by opting below 2 countermeasure alternatives.

Problem Statement: Currently the application is not using any encryption techniques to protect data.

Therefore, it has very high chances of data breaches or hacking. The probability of data breaches is 35%.

The cost of losing data will be measured in terms of cost of its reproduction which is estimated as 8.

Countermeasure alternative 1 and 2:

1. Add software to run vulnerability scans every fortnight. This option will reduce the risk to 3% and is estimated to cost 2.

2. Add hash tables, salt algorithms, or cryptography to protect your critical data. This option will reduce the risk to 2% and is estimated to cost 3.

$$RRL_1 = (0.35 \times 8 - 0.03 \times 8)/2 = (2.8 - 0.24)/2 = 1.28$$

$$RRL_2 = (0.35 \times 8 - 0.02 \times 8)/3 = (2.8 - 0.16)/3 = 0.88$$

Since $RRL_1 > RRL_2$ which indicates a cost-effective risk reduction measure. The first alternative is the better option.

d) Performance Risk

Risk Reduction Tactic: Reduce Risk Likelihood by opting below 2 countermeasure alternatives.

Problem Statement: Currently the system has a risk of low performance where transactional pages take longer than 2 seconds to load. The probability of violating this threshold is 24%. The cost of slow response times is measured in user drop-off which is estimated as 7.

Countermeasure alternative 1 and 2:

1. Distribute traffic with a Load Balancer. In this way data will be transmitted fast to the server. This option will reduce the risk to 1% and is estimated to cost 1.
2. Deliver content faster by client-side local caching. This option will reduce the risk to 3% and is estimated to cost 2.

$$RRL_1 = (0.24 \times 7 - 0.01 \times 7)/1 = (1.68 - 0.07)/1 = 1.61$$

$$RRL_2 = (0.24 \times 7 - 0.03 \times 7)/2 = (1.68 - 0.21)/2 = 0.73$$

Since $RRL_1 > RRL_2$ which indicates a cost-effective risk reduction measure. The first alternative is the better option.

e) Communication Risk

Risk Reduction Tactic: Reduce Risk Likelihood by opting below 2 countermeasure alternatives.

Problem Statement: There are high chances of communication failure between the App and Database server in mobile environments. The probability of data unavailability is 40%. The cost of unavailability will be measured in terms of system usability loss which is estimated as 8.

Countermeasure alternative 1 and 2:

1. Implement an Offline-First PWA mode to cache the grocery list locally. This option will reduce the risk to 1% and is estimated to cost 3.
2. Implement background synchronization that queues queries when offline. This option will reduce the risk to 2% and is estimated to cost 2.

$$RRL_1 = (0.40 \times 8 - 0.01 \times 8)/3 = (3.2 - 0.08)/3 = 1.04$$

$$RRL_2 = (0.40 \times 8 - 0.02 \times 8)/2 = (3.2 - 0.16)/2 = 1.52$$

Since $RRL_2 > RRL_1$ which indicates a cost-effective risk reduction measure. The second alternative is the better option.

Appendix

Task	Description	Time Taken(hours)
Task 1	Identifying and finding inconsistencies in vision document	1
Task 2	Documenting conflicts	2
Task 3	Conflict resolution	2
Task 4	Conflict evaluation	2
Task 5	Risk management	3
Total		10